

CO3-ASSESSMENT TOOL -2

CSA-0603-DESIGN AND ANALYSIS OF ALGORITHMS

CH.V.Ramayogi-192424319

Date:5-08-2026

Title:Stock Market Analysis System Using Quick Sort Algorithm

Description of the Real-World Problem

In today's stock market, millions of stock prices are updated every second. Investors and traders need these prices to be sorted quickly so that they can identify the highest-performing stocks, lowest-performing stocks, and market trends.

For example, a stock trading application such as NSE, BSE, or any online trading platform continuously receives live stock prices. If these prices are not sorted efficiently, users may experience delays while viewing market information.

Therefore, an efficient sorting algorithm is required to process dynamic stock market data in real time.

Problem Statement

Design a Stock Market Analysis System that sorts stock prices efficiently using the **Quick Sort Algorithm**.

The system should:

- Accept multiple stock prices.
- Sort them in ascending or descending order.
- Handle large datasets efficiently.
- Reduce processing time.
- Analyze average and worst-case performance.
- Improve performance by selecting an optimal pivot.

Algorithm Used

Quick Sort Algorithm

Quick Sort is a Divide and Conquer sorting algorithm.

It works in three steps:

1. Select a Pivot element.

2. Partition the array into
 - Elements smaller than pivot
 - Pivot
 - Elements greater than pivot
3. Recursively apply Quick Sort on left and right partitions.

Pseudo Code

QuickSort(arr, low, high)

if(low < high)

 pivot = Partition(arr, low, high)

 QuickSort(arr, low, pivot-1)

 QuickSort(arr, pivot+1, high)

Justification for Choosing the Algorithm (Why Quick Sort?)

Quick Sort is selected because:

- Very fast for large datasets.
- Average time complexity is $O(n \log n)$.
- Uses Divide and Conquer.
- Efficient cache performance.
- Widely used in real-world applications.
- Suitable for real-time stock price sorting.

Although Merge Sort guarantees $O(n \log n)$, Quick Sort generally performs faster in practice due to lower memory usage.

Divide and Conquer Principle

Quick Sort follows Divide and Conquer.

Divide

Choose one pivot.

Divide the array into two parts.

Conquer

Recursively sort both partitions.

Combine

No extra combining is required because partitioning itself places the pivot in its correct position.

Working Example (Manual Execution)

Suppose the stock prices are

[85, 45, 70, 30, 95, 60]

Choose Pivot = **60**

Step 1

Original

85 45 70 30 95 60

Partition

45 30 |60| 85 70 95

Left Side

45 30

Pivot = 30

Sorted

30 45

Right Side

85 70 95

Pivot = 95

85 70 |95|

Now sort

85 70

Pivot = 70

Sorted

70 85

Final Sorted Array

30 45 60 70 85 95

Pivot Selection Issues

Quick Sort performance depends heavily on pivot selection.

Good Pivot

- Middle element
- Random element
- Median of Three

Produces balanced partitions.

Average complexity:

$O(n \log n)$

Bad Pivot

Always selecting

- First element
- Last element

on already sorted arrays produces highly unbalanced partitions.

Worst complexity:

$O(n^2)$

Optimized Solution

Instead of always selecting the first or last element,

Use:

Random Pivot

or

Median of Three Pivot

Benefits:

- Reduces chances of worst-case.
- Better partition balance.
- Faster execution.
- Improves real-time performance.

Complexity Analysis

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

Space Complexity

$O(\log n)$

(recursion stack)

Applications

Quick Sort is widely used in

- Stock Market Analysis
- Online Trading Platforms
- Database Management Systems
- Banking Systems
- Data Analytics
- Search Engines
- Financial Applications

- Scientific Computing

Advantages

- Very fast in practice.
- Average complexity $O(n \log n)$.
- In-place sorting.
- Low memory usage.
- Efficient for large datasets.
- Suitable for real-time applications.

Limitations

- Worst-case complexity $O(n^2)$.
- Performance depends on pivot selection.
- Recursive implementation may increase stack usage.
- Not a stable sorting algorithm.

Comparison with Other Algorithms

Algorithm	Best	Average	Worst	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Sample Python Program

=====

```

# Stock Market Analysis System using Quick Sort

# =====

# Function to partition the array

def partition(stock, low, high):

    pivot = stock[high]

    i = low - 1

    for j in range(low, high):

        if stock[j] <= pivot:

            i += 1

            stock[i], stock[j] = stock[j], stock[i]

    stock[i + 1], stock[high] = stock[high], stock[i + 1]

    return i + 1

# Quick Sort Function

def quick_sort(stock, low, high):

    if low < high:

        pivot_index = partition(stock, low, high)

        quick_sort(stock, low, pivot_index - 1)

        quick_sort(stock, pivot_index + 1, high)

# Main Program

print("=" * 50)

print("    STOCK MARKET ANALYSIS SYSTEM")

print("    USING QUICK SORT")

print("=" * 50)

n = int(input("\nEnter the number of stock prices: "))

stock_prices = []

```

```

print("\nEnter the stock prices:")

for i in range(n):

    price = float(input(f"Stock {i + 1}: "))

    stock_prices.append(price)

# Perform Quick Sort

quick_sort(stock_prices, 0, len(stock_prices) - 1)

print("\n" + "=" * 50)

print("Sorted Stock Prices (Ascending Order)")

print("=" * 50)

for price in stock_prices:

    print(price)

print("\nHighest Stock Price :", stock_prices[-1])

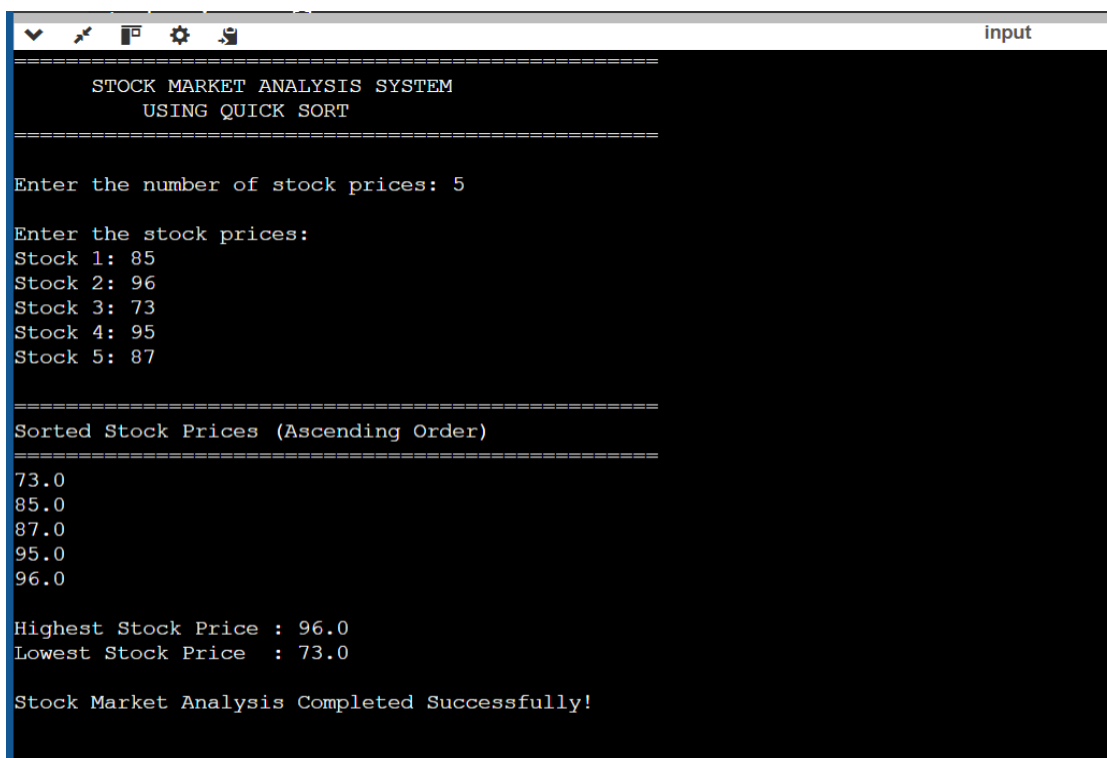
print("Lowest Stock Price :", stock_prices[0])

print("\nStock Market Analysis Completed Successfully!")

```

Output Screenshot

Sample Input



```

=====
STOCK MARKET ANALYSIS SYSTEM
USING QUICK SORT
=====

Enter the number of stock prices: 5

Enter the stock prices:
Stock 1: 85
Stock 2: 96
Stock 3: 73
Stock 4: 95
Stock 5: 87

=====
Sorted Stock Prices (Ascending Order)
=====
73.0
85.0
87.0
95.0
96.0

Highest Stock Price : 96.0
Lowest Stock Price  : 73.0

Stock Market Analysis Completed Successfully!

```


Enter number of stock prices:

5

Enter stock prices:

85

96

73

95

87

Sample Output

Sorted Stock Prices:

73.0

85.0

87.0

95.0

96.0

Highest Stock Price:96.0

Lowest Stock Price:73.0

Conclusion

Quick Sort is one of the fastest and most efficient sorting algorithms for stock market analysis systems due to its average time complexity of $O(n \log n)$ and in-place sorting capability. By applying the Divide and Conquer technique, it efficiently processes large volumes of dynamic stock data. However, poor pivot selection can degrade performance to $O(n^2)$. Using optimized pivot selection methods such as Random Pivot or Median-of-Three significantly improves performance and minimizes the chances of worst-case scenarios. Hence, Quick Sort is a practical and widely used algorithm for real-time stock market applications.